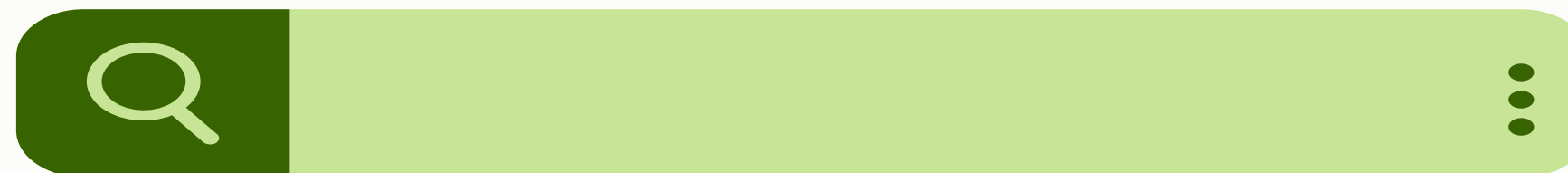




Reflection

Presented by Sara Abbasghorbani

Advanced Programming - Spring 2026 - Dr Vahidi Asl



○○○ What will we learn in this session?

1) Reflection in Java

2) Key classes of Reflection

3) Inspecting Classes

4) Some Examples

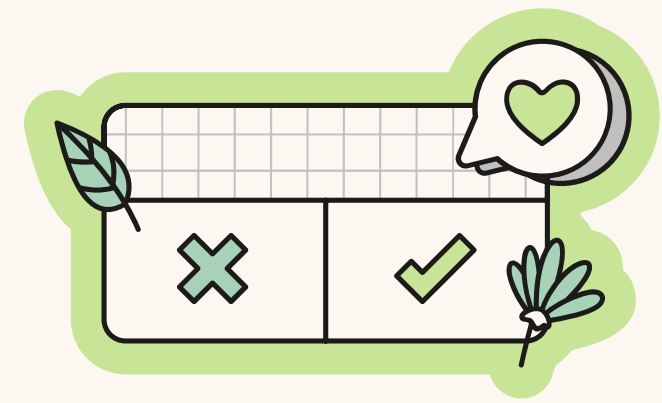
5) Applications of Reflection

6) When Use Reflection



Question1

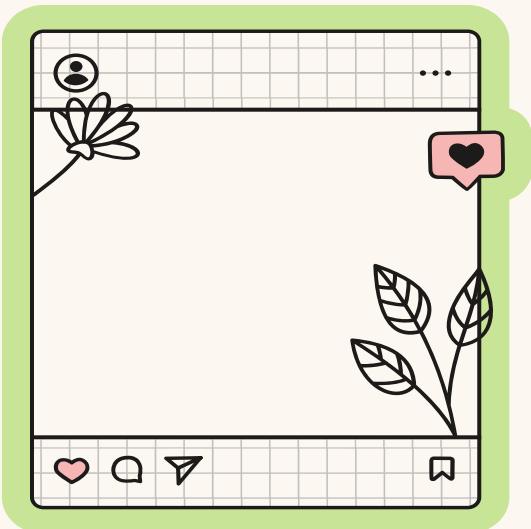
Reflection meaning?



Answer

Reflection meaning in Java:

The program can examine itself and modify itself at runtime. The code can be read, inspected, and changed during execution.





Question2

Why Use Reflection?



Answer

Reflection is particularly useful in scenarios where:

Dynamic Behavior: You need to dynamically load classes, invoke methods, or access fields without hardcoding their names.

Framework Development: Many popular frameworks (e.g., Spring, Hibernate) rely on Reflection to inject dependencies, map objects to database tables, or handle annotations.

Testing: Reflection can be used to test private methods or fields without modifying the original code.

Plugins and Extensibility: Reflection enables you to load and execute code from external libraries or plugins dynamically.

Reflection in Java



Reflection is a feature in the Java programming language. It allows an executing Java program to examine or "introspect" upon itself, and manipulate internal properties of the program. For example, it's possible for a Java class to obtain the names of all its members and display them.

The ability to examine and manipulate a Java class from within itself may not sound like very much, but in other programming languages this feature simply doesn't exist. For example, there is no way in a Pascal, C, or C++ program to obtain information about the functions defined within that program.

One tangible use of reflection is in JavaBeans, where software components can be manipulated visually via a builder tool. The tool uses reflection to obtain the properties of Java components (classes) as they are dynamically loaded.

Reflection in Java

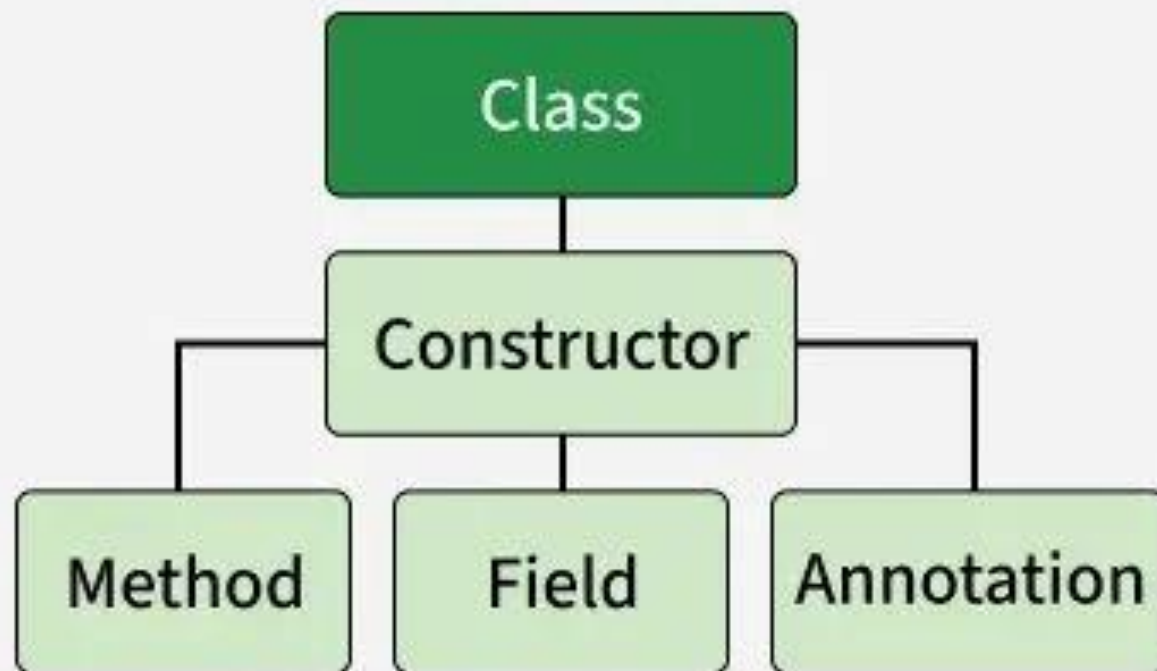


Reflection in Java allows a program to inspect and manipulate classes, methods, fields, and constructors at runtime, even when their details are unknown at compile time. It is provided through the `java.lang.reflect` package and is widely used in frameworks and libraries.

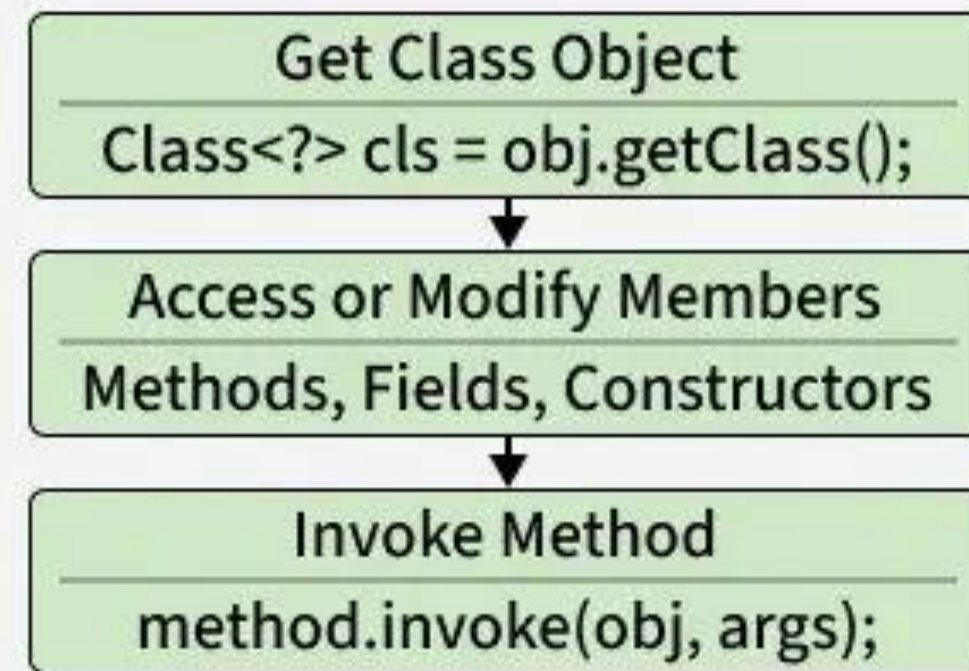
- 1) To examine class metadata (methods, fields, constructors) during runtime.
- 2) To create objects and invoke methods dynamically.
- 3) To access and modify fields or methods whose names are determined at runtime.
- 4) To build flexible, extensible frameworks that work with unknown classes.

Reflection in Java

Class Structure



Reflection Workflow



Access Private Members



Bypass Access Control





Question3

How Does Reflection Work?



Answer

Reflection in Java is part of the `java.lang.reflect` package.

The key classes and interfaces include:

Class: Represents the metadata of a class.

Field: Represents a field (variable) of a class.

Method: Represents a method of a class.

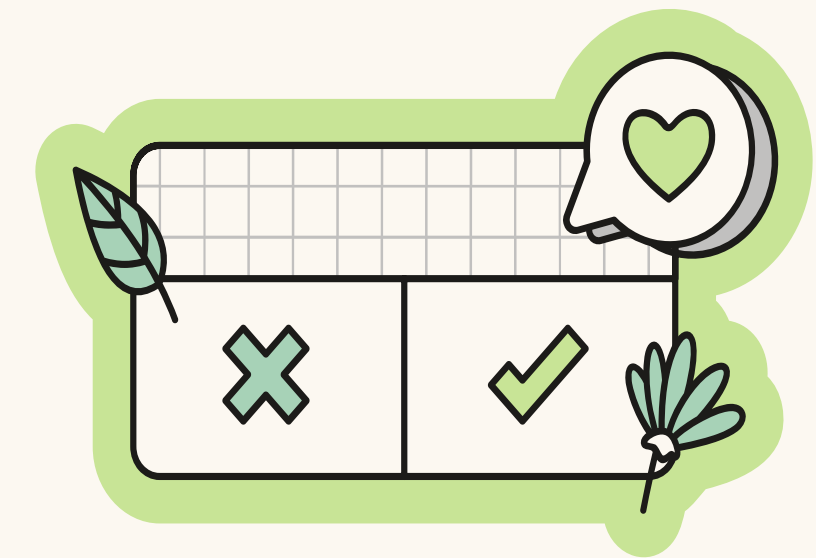
Constructor: Represents a constructor of a class.

1. Accessing Class Metadata

```
public class ReflectionDemo {  
    public static void main(String[] args) {  
        try {  
            // Load the class  
            Class<?> clazz = Class.forName("java.util.ArrayList");  
  
            // Print class name  
            System.out.println("Class Name: " + clazz.getName());  
  
            // Print superclass name  
            System.out.println("Superclass: " + clazz.getSuperclass());  
  
            // Print implemented interfaces  
            Class<?>[] interfaces = clazz.getInterfaces();  
            System.out.println("Implemented Interfaces:");  
            for (Class<?> iface : interfaces) {  
                System.out.println(iface.getName());  
            }  
        } catch (ClassNotFoundException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

```
Class Name: java.util.ArrayList  
Superclass: java.util.AbstractList  
Implemented Interfaces:  
java.util.List  
java.util.RandomAccess  
java.lang.Cloneable  
java.io.Serializable
```

Output



2. Accessing and Modifying Fields

```
import java.lang.reflect.Field;

class Person {
    private String name = "John Doe";
}

public class FieldAccessDemo {
    public static void main(String[] args) {
        try {
            Person person = new Person();

            // Access the class
            Class<?> clazz = person.getClass();

            // Access the private field
            Field nameField = clazz.getDeclaredField("name");

            // Make the field accessible
            nameField.setAccessible(true);

            // Get the value of the field
            String name = (String) nameField.get(person);
            System.out.println("Original Name: " + name);

            // Modify the value of the field
            nameField.set(person, "Jane Doe");
            System.out.println("Modified Name: " + nameField.get(person));
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Output:

```
Original Name: John Doe
Modified Name: Jane Doe
```


3. Invoking Methods Dynamically

```
import java.lang.reflect.Method;

class Calculator {
    private int add(int a, int b) {
        return a + b;
    }
}

public class MethodInvokeDemo {
    public static void main(String[] args) {
        try {
            Calculator calculator = new Calculator();

            // Access the class
            Class<?> clazz = calculator.getClass();

            // Access the private method
            Method addMethod = clazz.getDeclaredMethod("add", int.class, int.class);

            // Make the method accessible
            addMethod.setAccessible(true);

            // Invoke the method
            int result = (int) addMethod.invoke(calculator, 5, 10);
            System.out.println("Result: " + result);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Output:

Result: 15

4. Creating Objects Dynamically

```
import java.lang.reflect.Constructor;

class User {
    private String username;

    public User(String username) {
        this.username = username;
    }

    public String getUsername() {
        return username;
    }
}

public class ConstructorDemo {
    public static void main(String[] args) {
        try {
            // Access the class
            Class<?> clazz = User.class;

            // Access the constructor
            Constructor<?> constructor = clazz.getConstructor(String.class);

            // Create a new instance
            User user = (User) constructor.newInstance("john_doe");

            // Print the username
            System.out.println("Username: " + user.getUsername());
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Output:

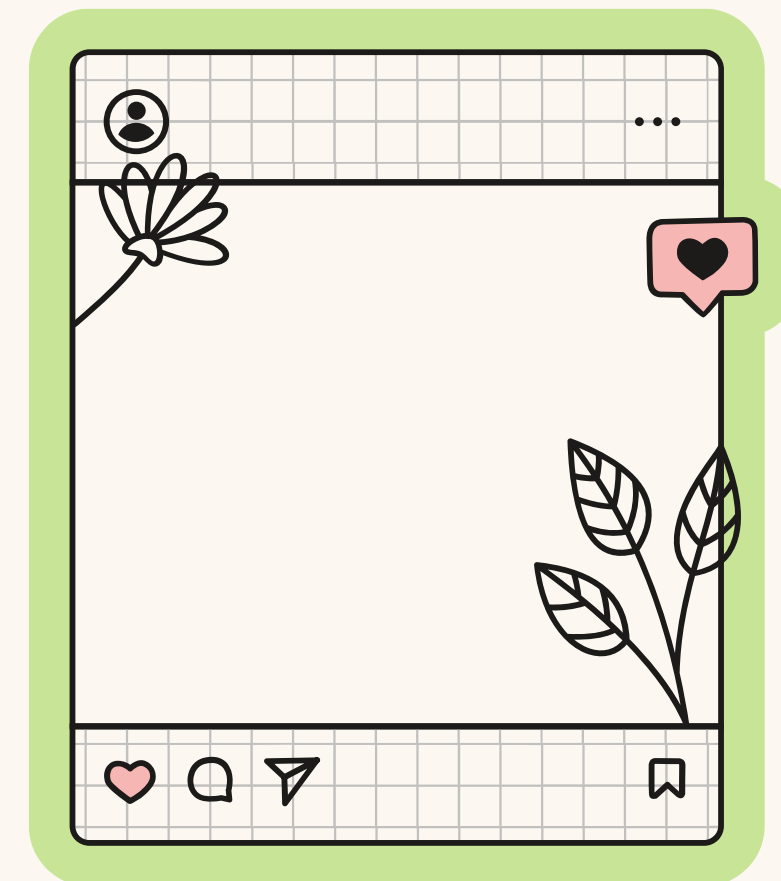
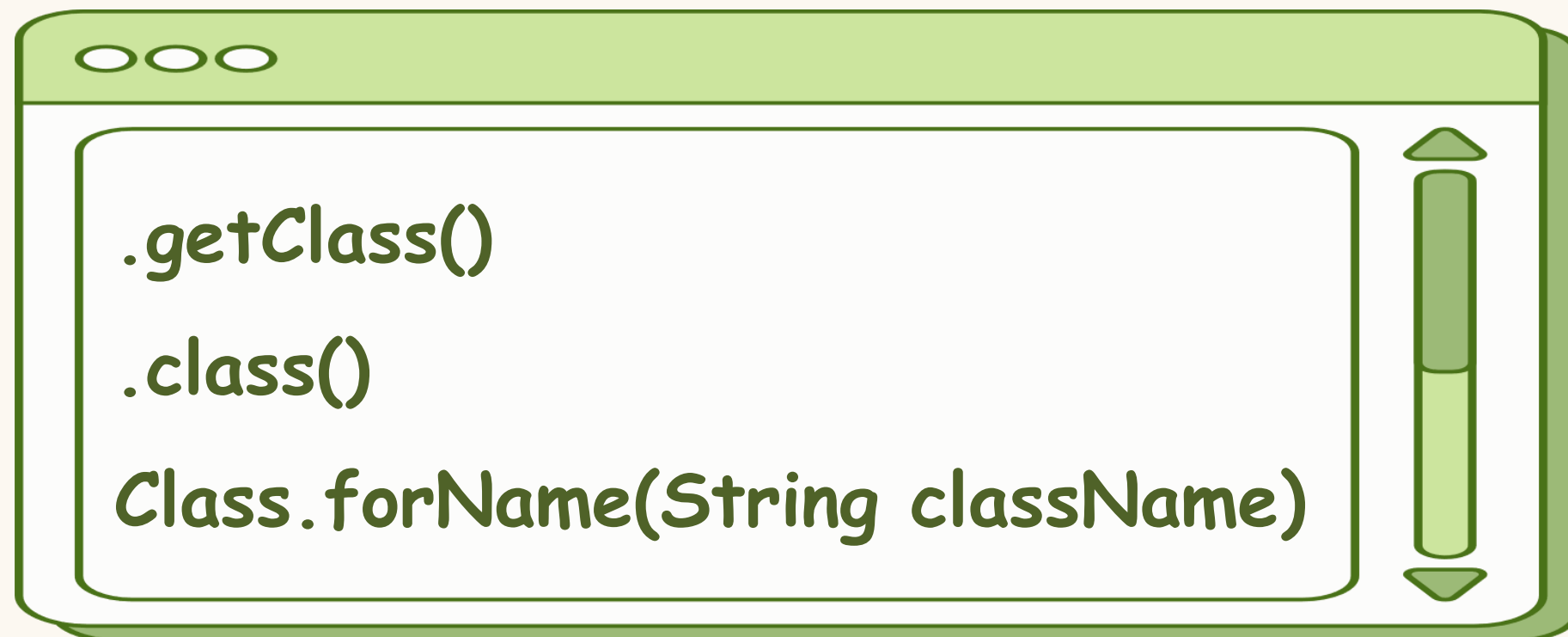
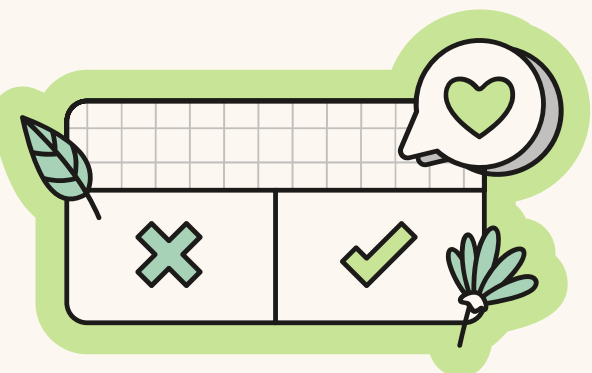
Username: john_doe

Reflection vs Normal Code

Feature	Reflection	Normal Code
Compile-time Safety	No	Yes
Performance	Slower	Faster
Performance	High	Low
Security	Lower	Higher

Inspecting Classes

The foundational step in utilizing Java's Reflection API is obtaining an instance of the `Class` class. This object is like a blueprint that provides metadata about a particular class, including its constructors, methods, fields, and more. Once you have a `Class` instance, you can introspect various aspects of the corresponding class. Here's how you can retrieve such an instance : `.getClass()` , `.class()` , `Class.forName(String className)`



```
1 ▶ public class Information {  
2 ▶     public static void main(String[] args){  
3         String word = "Hello Reflection !" ;  
4         Class<?> cls = word.getClass() ;  
5         System.out.println(cls.getName());  
6     }  
7 }
```

Information x

"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaa
java.lang.String

○○○ Getting Class Information

Using the `.getClass()` method
All Java objects inherit the `getClass()` method from the `java.lang.Object` class. This method returns a `Class` object representing the runtime class of the instance. It's an easy and direct approach when you have an object instance.

○○○ Getting Class Information

```
1 ▶ public class Information {  
2 ▶     public static void main(String[] args){  
3         Class<?> intClass = int.class ;  
4         Class<?> listClass = java.util.ArrayList.class ;  
5         System.out.println(intClass);  
6         System.out.println(listClass);  
7     }  
8 }
```

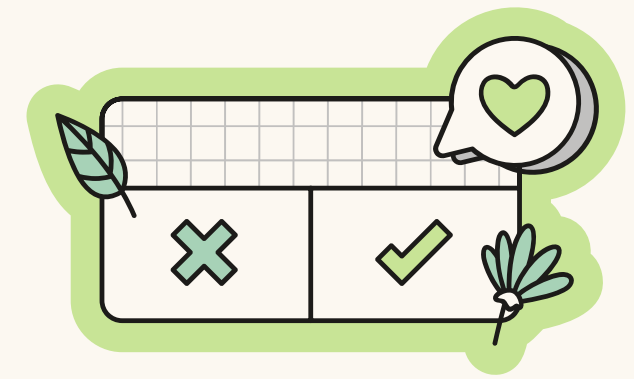
Information x

```
"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Pro  
int  
class java.util.ArrayList
```

○○○

Using .class on a class name
Sometimes, you might want to get the Class object without instantiating an object. In such cases, using .class syntax on class names or primitive data types provides a straightforward solution.

Getting Class Information



```
1 ▶ public class Information {
2 ▶     public static void main(String[] args){
3         try {
4             Class<?> DynamicClass = Class.forName( className: "java.util.HashMap" ) ;
5             System.out.println(DynamicClass.getSimpleName());
6         }catch(ClassNotFoundException exception){
7             exception.printStackTrace();
8         }
    }
```

Information x

"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA\bin\idea-agent.jar" -classpath C:\Program Files\Java\jdk-21\bin\java.exe HashMap

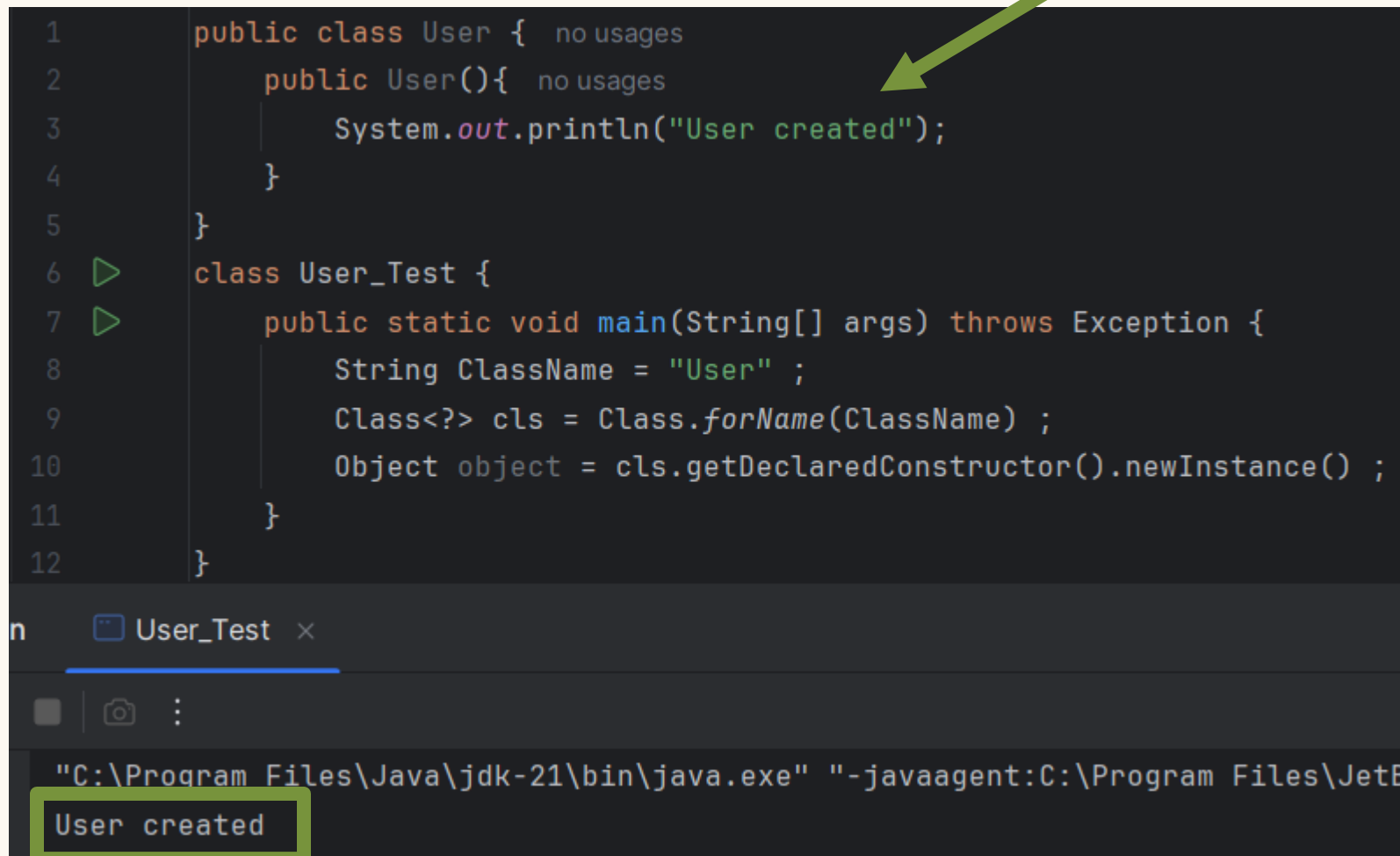


Using `Class.forName(String className)`
Another powerful tool in your reflection arsenal is the `Class.forName()` method. This method allows you to dynamically load and link a class using its fully qualified name.



Some Examples

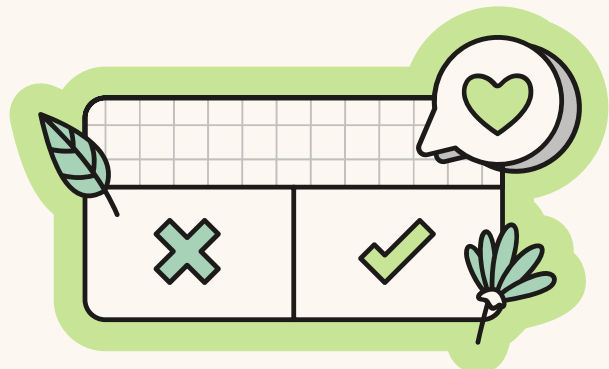
Dynamic Object Creation



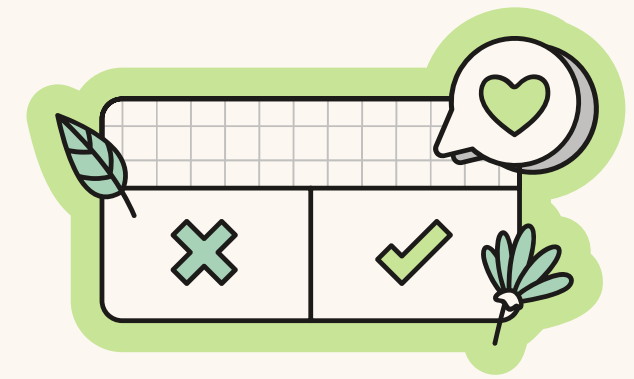
```
1 public class User { no usages
2     public User(){ no usages
3         System.out.println("User created");
4     }
5 }
6 class User_Test {
7     public static void main(String[] args) throws Exception {
8         String ClassName = "User" ;
9         Class<?> cls = Class.forName(ClassName) ;
10        Object object = cls.getDeclaredConstructor().newInstance() ;
11    }
12 }
```

Output: User created

Reflection allows creating objects at runtime when the class name is not known at compile time. This is useful in plugin systems and frameworks.



Building a simple JSON mapper



```
1  import java.lang.reflect.Field;
2  import java.util.HashMap;
3  import java.util.Map;
4
5  public class Person { 3 usages
6      public String name ; 1 usage
7      public int age ; 1 usage
8  }
9  class Person_Test {
10     public static void main(String[] args) throws Exception{
11         Map<String , Object> json = new HashMap<>() ;
12         json.put("name" , "Sara") ;
13         json.put("age" , 21) ;
14         Person person = new Person() ;
15         Field[] fields = Person.class.getDeclaredFields() ;
16         for (Field field : fields) {
17             Object object = json.get(field.getName());
18             field.setAccessible(true);
19             field.set(person, object);
20         }
21         System.out.println(person.name + " " + person.age);
22     }
23 }
```

Person_Test x

"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program File
Sara 21

Reflection is used in libraries
like Jackson to map JSON
data to Java objects
dynamically.

Annotation-Based Execution

```
1 import java.lang.annotation.Retention;
2 import java.lang.annotation.RetentionPolicy;
3 import java.lang.reflect.InvocationTargetException;
4 import java.lang.reflect.Method;
5
6 @Retention(RetentionPolicy.RUNTIME) 2 usages
7 @interface Test {}
8 class My_Tests { 3 usages
9     @Test no usages
10     public void test1() {
11         System.out.println("Test 1 run");
12     }
13     public void test2() { no usages
14         System.out.println("Test 2 run");
15     }
16 }
17 public class Write_Notation {
18     public static void main(String[] args) throws InvocationTargetException, IllegalAccessException {
19         My_Tests myTests = new My_Tests();
20         Method[] methods = My_Tests.class.getDeclaredMethods();
21         for(Method method : methods){
22             if(method.isAnnotationPresent(Test.class))
23                 method.invoke(myTests);
24         }
25     }
26 }
27 }
```

Run Write_Notation x

"C:\Program Files\Java\jdk-21\bin\java.exe"
Test 1 run

Frameworks use
reflection to scan
annotations and
execute methods
automatically.

Applications of Reflection

1) Dependency Injection Frameworks (e.g., Spring)

Frameworks like Spring heavily rely on Reflection to scan your application, find annotations like `@Autowired`, and automatically instantiate and inject dependencies (Beans) at runtime without requiring explicit new keywords.

2) Object-Relational Mapping / ORM (e.g., Hibernate)

Tools like Hibernate use Reflection to map database tables to Java objects. They can directly inject database values into private fields of your classes, completely bypassing the need for public setter methods.

3) Testing Frameworks (e.g., JUnit)

.When you run tests, JUnit uses Reflection to inspect your test classes, identify all methods marked with the `@Test` annotation, and invoke them dynamically at runtime

4) Data Serialization & Deserialization (e.g., Jackson, Gson)

Libraries that convert Java objects to JSON (and vice versa) use Reflection to inspect class structures. They read private field names and values to generate JSON keys, and dynamically construct Java objects when parsing JSON files.



Question4

When to Use Reflection (and When Not To)



Answer

Advantages(can use):

1. **Flexibility:** Reflection allows you to write dynamic and generic code.
2. **Extensibility:** It's essential for building frameworks and libraries.
3. **Testing:** It enables you to test private methods and fields without modifying the original code.

Disadvantages(can't use):

1. **Performance Overhead:** Reflection is slower than direct method calls or field access.
2. **Security Risks:** It can bypass access modifiers, potentially exposing sensitive data.
3. **Complexity:** Reflection can make the code harder to read and debug.

○○○ Note

- Reflection is powerful but expensive
- Avoid using it in performance-critical code
- Prefer normal object access when possible
- Use reflection mainly in frameworks

Resources

